
Optimal Brain Compression for VGGNet on CIFAR

Muhammad Bin Tariq Muhammad Sabeer Asad Raahim A Samad Poonawala

Abstract

We explore the Lottery Ticket Hypothesis by implementing pruning techniques on the VGG11 architecture trained on the CIFAR dataset. Pruning is a viable compression technique that yields a smaller, faster subnetwork with similar benchmarks as the dense model. Our scaling law analysis for quantization reveals that Post-Training Quantization is more robust than Quantization Aware Training at multiple bit-widths, despite QAT's higher overall accuracy. Lastly, we investigate the efficacy of knowledge distillation techniques such as FitNets and Contrastive Representation Distillation in transferring knowledge from a larger teacher model to a smaller student model. All our code can be found [here](#).

1. Introduction

Dating back to 1989 when Optimal Brain Damage (OBD) (LeCun et al., 1989) was first introduced by LeCun et. al., network pruning has emerged as a prominent technique for model compression. The emergence of deep neural networks which are often over-parameterized (LeCun et al., 1989; Han et al., 2015) has presented the significant challenge of deploying these models on resource-constrained devices. The goal is to create smaller, more efficient models that retain the high performance of their larger counterparts. This paper explores three fundamental pillars of model compression.

Pruning techniques aim to address this challenge by removing redundant or less important parameters from the network, thereby reducing its size and computational requirements without significantly compromising performance. Meanwhile, Quantization reduces the numerical precision of the model's parameters (weights) and/or activations. By converting 32-bit floating-point numbers ($FP32$) to lower bit-width representations (e.g., $INT8$ or $FP16$), quantization significantly reduces the memory footprint and can accelerate inference on compatible hardware. Lastly, first introduced by Hinton et al. (Hinton et al., 2015), Knowledge Distillation (KD) is a process where a compact "student" model is trained to mimic the behavior of a larger, pre-trained "teacher" model. The student learns from the

teacher's softened probability distributions or internal feature representations, capturing a richer source of information than hard labels alone.

2. Methodology

All of our experiments are conducted on the CIFAR dataset. We evaluate different methods for each technique to obtain a holistic perspective on which model compression technique suits well in different tasks.

2.1. Optimal Brain Surgeon: Pruning

Our experimental setup closely follows the methods outlined by Han et. al. in (Han et al., 2015), where we perform magnitude-based unstructured pruning, and channel-wise structured pruning on the Visual Geometry Group Network (VGG11) architecture trained on the CIFAR-10 dataset.

The experiments follow a three-step process: (1) training the VGG11 model to establish a baseline accuracy, (2) applying pruning techniques to reduce the number of parameters, and (3) fine-tuning (FT) the pruned model to recover any loss in accuracy.

Our implementation was inspired by MIT Han Lab's course materials that can be found over [here](#).

2.1.1. UNSTRUCTURED PRUNING

Using an L2-norm magnitude-based approach, we prune individual weights in the network focusing on those that contribute the least to the overall performance. By visualizing the weight distributions and performing sensitivity analysis, we determine optimal pruning thresholds for each layer. After setting these sparsity levels, we prune the weights of each layer accordingly and fine-tune the model to regain any loss in accuracy. We call this pruned model VGG11-UBP (Unstructured Brain Pruned).

2.1.2. STRUCTURED PRUNING

We conduct channel-wise pruning by evaluating the importance of each channel via the Frobenius norm of the weight tensors. The channels were ranked based on their computed norms, and the least important channels were pruned to achieve the desired sparsity levels. Post-pruning, we fine-

tune the model to restore its performance. We refer to this pruned model as VGG11-SBP (Structured Brain Pruned).

2.1.3. FEATURE EXPLAINABILITY VIA GRAD-CAM

To visualize and understand the impact of pruning on feature extraction, we employ Gradient-weighted Class Activation Mapping (Grad-CAM) (Selvaraju et al., 2019). Grad-CAM generates heatmaps that highlight the regions of input images that are most influential in the model’s decision-making process. By comparing the Grad-CAM outputs of the original and pruned models, we can assess how pruning affects the model’s focus on relevant features.

2.2. Quantization

Our experimental framework uses PyTorch and the torchao library. We evaluate all methods on a VGG-11 model fine-tuned on the CIFAR-100 dataset against a 32-bit floating-point (FP32) baseline. Key metrics are top-1 accuracy (%), inference latency (ms), and model size (MB).

PTQ AND QAT

We compare PTQ, where a trained model is quantized directly, against QAT, where quantization is simulated during a few epochs of retraining. Both methods were applied to produce models in fp16, bf16, int8, and int4 precision.

SCALING LAW ANALYSIS

Using the accuracy results from the PTQ and QAT experiments, we plot accuracy versus bit-width (32, 16, 8, 4) for each method. This visualizes the performance degradation curve and highlights the precision levels at which accuracy becomes most sensitive.

MIXED-PRECISION QUANTIZATION

We evaluate three configurations to analyze the trade-offs of assigning precision on a layer-by-layer basis:

- **Uniform Quantization:** A baseline where all layers are quantized to INT8.
- **Simple Mixed-Precision:** The first and last layers are kept in FP32, while intermediate layers are quantized to INT8.
- **Adaptive Mixed-Precision:** Layers are kept in FP32 or quantized to INT8 based on their activation variance, a proxy for sensitivity. Layers with the highest variance remain in FP32.

EFFECT OF OUTLIERS

We investigate how extreme values impact quantization quality by comparing several strategies applied to the model’s weights at 8-bit and 4-bit precision:

- **Uniform Quantization:** A standard linear mapping using the tensor’s min/max values.
- **Clipped Quantization:** The quantization range is set by a percentile of the distribution (99.9% or 99.0%), thereby clipping outliers.
- **Logarithmic Quantization:** A non-linear mapping that provides more resolution near zero.

2.3. Knowledge Distillation

Our focus is a deep dive into KD, where we compare various state-of-the-art techniques, including logit-matching, feature-based hints, and contrastive representation distillation.

We analyze the efficacy of these methods in transferring knowledge from a pre-trained VGG-16 as the teacher model (T) and a VGG-11 as the student model (S_T), trained from scratch. Both models were loaded from the chenyaofu/pytorch-cifar-models repository, specifically the cifar100_vgg16_bn (pretrained) and cifar100_vgg11_bn (from scratch) variants. These models are adapted for the 32x32 CIFAR-100 dataset, which significantly differs from the original VGG architecture designed for 224x224 ImageNet images. While the convolutional feature extractors remain identical, the classifier heads are modified.

For all student training, we employ a consistent optimization setup across 50 epochs. The optimizer is Stochastic Gradient Descent (optim.SGD) with a learning rate of 0.01, momentum of 0.9, and weight decay of 5e-4. No learning rate scheduler was used due to the less number of epochs due to compute constraints.

2.3.1. LOGIT MATCHING VARIANTS

In this task, we implemented and compared three distinct loss functions.

- **Basic Logit Matching (LM):** The classic KD method (Hinton et al., 2015), where the student is trained to match the teacher’s softened logits using a Kullback-Leibler (KL) divergence loss. We combine the KL divergence loss (with a temperature=4.0) and a standard nn.CrossEntropyLoss weighted by alpha=0.9.
- **Label Smoothing (LS):** regularization technique, not a true distillation method, which we use as a baseline

(Szegedy et al., 2016). It prevents overconfidence by softening the hard ground-truth labels. and does not use the teacher model. We modify the standard cross-entropy loss by setting the target label confidence to $1.0 - \epsilon$ and distributing ϵ among the other classes. We used `smoothing=0.1`.

- **Decoupled KD (DKD):** This method (Zhao et al., 2022) separates the distillation loss into two parts: one for the target class and one for the non-target classes, allowing for more granular control over the knowledge transfer. We used a `temperature=4.0`, an `alpha` (target weight) of 1.0, and a `beta` (non-target weight) of 8.0.

2.3.2. COMPARATIVE SOTA ANALYSIS

We compared the performance of students trained with three different paradigms of KD against an independently trained student (S_I).

- **Logit Matching (LM):** This uses the trained model from last section.
- **Hint-based (Hints):** This method, based on FitNets (Romero et al., 2015), uses an adapter layer to train the student’s intermediate feature maps - the final layer in our experiments, to mimic those of the teacher. The `HintLoss` function then computes an `nn.MSELoss` between the teacher and student feature maps at this layer, combined with a standard `nn.CrossEntropyLoss` on the logits. The hint loss was weighted with an `alpha=100.0`.
- **Contrastive Representation Distillation (CRD):** A more recent approach that uses a contrastive loss to align the learned representations of the teacher and student models in a shared embedding space (Tian et al., 2022). We use `temperature=0.07` which is then applied to these projected features. The final training loss was a weighted sum of `nn.CrossEntropyLoss` and this contrastive loss, specifically $\mathcal{L} = \mathcal{L}_{CE} + 0.8 \times \mathcal{L}_{CRD}$.

2.3.3. PROBABILITY DISTRIBUTION ALIGNMENT

To analyze distribution mimicry, we extracted the output logits for the teacher (T) and all four student models (S_{LM} , S_{Hint} , S_{CRD} , S_I) on 1000 samples from the validation set. We computed the probability distributions by applying a standard softmax function (i.e., temperature $T=1$) to these logits instead of computing over soft logits. We then calculated the average KL divergence between the teacher’s distribution and each of the four student distributions over these samples.

2.3.4. LOCALIZATION KNOWLEDGE TRANSFER

We used a standard GradCAM implementation to generate We investigate whether KD transfers the teacher’s ability to localize relevant image features. We use GradCAM (Selvaraju et al., 2017) to generate attention heatmaps for T , each S_D , and S_I . Following the methodology of Ojha et al. (Ojha et al., 2023), we quantify the similarity between the teacher’s heatmaps and each student’s heatmaps using the Structural Similarity Index (SSIM) and Mean Squared Error (MSE) on a fixed set of 10 query images from the validation set.

2.3.5. CHECKING FOR COLOR INVARIANCE WITH CRD

This experiment was conducted in three stages. First, we created a color-invariant teacher (T_{color}) by fine-tuning the pre-trained VGG-16 on the CIFAR-100 training set using aggressive color augmentations: `transforms.ColorJitter(brightness=0.8, contrast=0.8, saturation=0.8, hue=0.2)`. Second, we trained a VGG-11 student from scratch using the CRD methodology, using T_{color} as the teacher but training the student on the *standard* training dataset (with regular augmentations). Third, we evaluated this distilled student (S_{CRD}), along with all other previously trained student models, on two test sets: the standard test set and a color-jittered version created using the same `ColorJitter` parameters.

2.3.6. TESTING THE EFFICACY OF A LARGER TEACHER

Finally, we explore the “larger teacher” hypothesis, which posits that larger teachers produce better students. We compare the performance of two VGG-11 students trained with Logit Matching, one using the VGG-16 teacher (from previous experiments) and another using a larger VGG-19 teacher. This is inspired by work from (Cho & Hariharan, 2019).

3. Results

3.1. Optimal Brain Surgeon: Pruning

Model	Accuracy (%)
VGG11 (Baseline)	92.95
VGG11-UBP (No FT)	80.62
VGG11-SBP (No FT)	20.00
VGG11-UBP (FT)	92.71
VGG11-SBP (FT)	91.19

Table 1. Accuracy of Baseline and Pruned Models

3.1.1. UNSTRUCTURED PRUNING

Before pruning, we visualized the weight distribution of each layer to understand the spread and magnitude of weights (see Figure 1, more details in the appendix). Post unstructured pruning, we observed a significant shift in the weight distribution, with many weights being set to zero (see Figure 2). The sparsity scan (Figure 4) helped us determine optimal sparsity levels for each layer before finalizing the pruning process. Naturally, layers that would exhibit lesser sensitivity to pruning were assigned higher sparsity levels.

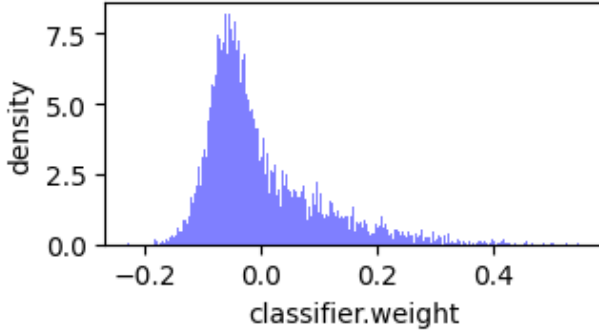


Figure 1. Weight Distribution of classification layer before Pruning

We observe that unstructured pruning retains a high level of accuracy even after significant "brain damage". As shown in Table 1, the VGG11-UBP model yields an accuracy of 80.62% without fine-tuning, which improves to 92.71% after fine-tuning, closely matching the baseline accuracy of 92.95%. This indicates that unstructured pruning effectively removes redundant weights while preserving the model's performance. Our results align with findings from Han et. al. (Han et al., 2015), demonstrating the efficacy of magnitude-based pruning techniques.

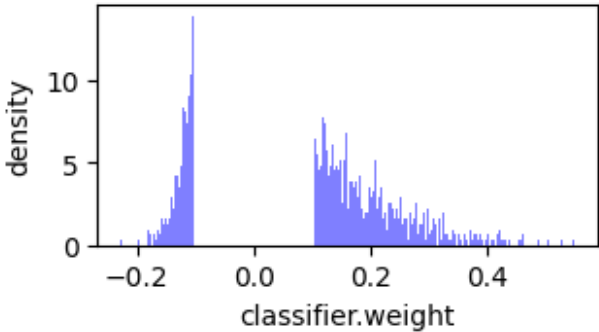


Figure 2. Weight Distribution of classification layer after Unstructured Pruning

3.1.2. STRUCTURED PRUNING

In order to perform structured pruning, we utilized the previously visualized weight distributions in tandem with a sparsity scan (Figure 4) of each layer to determine the optimal channels to prune (given the channels were ranked based on their Frobenius norm). Post structured pruning, we observed a significant reduction in the number of channels, leading to a more compact model: refer to Table 2 for details (7.12 MiB size compared to 35.20 MiB of the baseline and 2.83 MiB of unstructured pruning). However, this came at the cost of a more pronounced drop in accuracy (20.00% without fine-tuning), which improved to 91.19% after fine-tuning, as shown in Table 1. This indicates that while structured pruning is effective in reducing model size and complexity, it may require more extensive fine-tuning to recover performance compared to unstructured pruning.

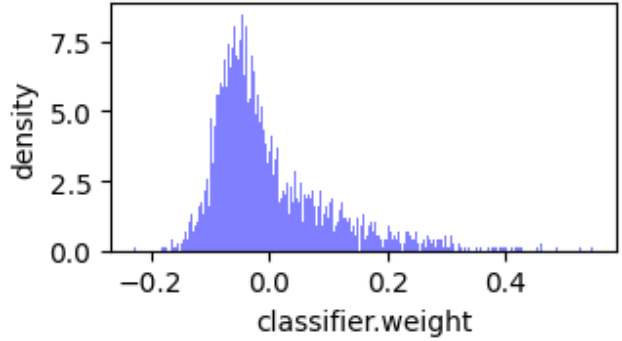


Figure 3. Weight Distribution of classification layer after Structured Pruning

Figure 3 illustrates the weight distribution of the classification layer after structured pruning. Note that the distribution seems very similar to that of the dense layer. This is because structured pruning removes entire channels, leading to a reduction in the number of filters rather than individual weights. As a result, the remaining weights still exhibit a distribution similar to that of the original dense layer, but with fewer overall parameters.

3.1.3. GRAD-CAM VISUALIZATION AND COMPARISON

Figure 9 presents the Grad-CAM visualizations for an example image of a frog across the baseline VGG11 model, VGG11-UBP, and VGG11-SBP. Each model effectively highlights the frog in the image. However, the baseline pruned model's Grad-CAM appears more focused and precise (indicated by a slightly darker red region), with the VGG11-SBP model showing the most focused attention on the frog.

Moreover, Table 2 shows that the VGG11-SBP model achieves the lowest latency (10.4 ms) compared to both

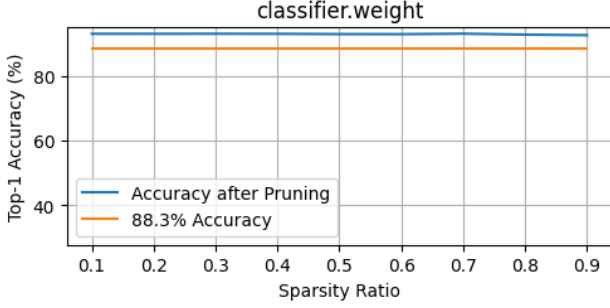


Figure 4. Sparsity scan of classification layer before final pruning is performed.

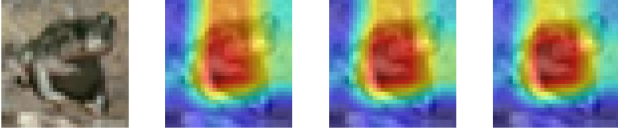


Figure 5. **Left-to-right:** Original Image (a frog), Grad-CAM of Baseline VGG11, Grad-CAM of VGG11-UBP, Grad-CAM of VGG11-SBP

the baseline (25.0 ms) and VGG11-UBP (26.0 ms). This indicates that structured pruning not only reduces model size but also enhances inference speed.

We also observed the effect of varying sparsity levels on model accuracy, as depicted in Figure 6. The accuracy remains relatively stable up to a sparsity level of 0.50, after which it starts to drop more significantly. However, this drop is mitigated by fine-tuning the model post-pruning, allowing it to recover much of its lost accuracy.

Model	Latency	Size (MiB)	Parameters (M)
VGG11 (Baseline)	25.0	35.20	9.23
VGG11-UBP (FT)	26.0	9.98	2.83
VGG11-SBP (FT)	10.4	7.12	1.87

Table 2. Statistical comparison of Baseline and Pruned Models’ inference time, size, and number of parameters.

3.2. Quantization

3.2.1. PTQ vs. QAT COMPARISON

The performance of all PTQ and QAT models is summarized in Table 3.

Accuracy remained stable across nearly all methods, with most staying within 1% of the 60.91% FP32 baseline.

Model Size was effectively reduced by all methods. PTQ

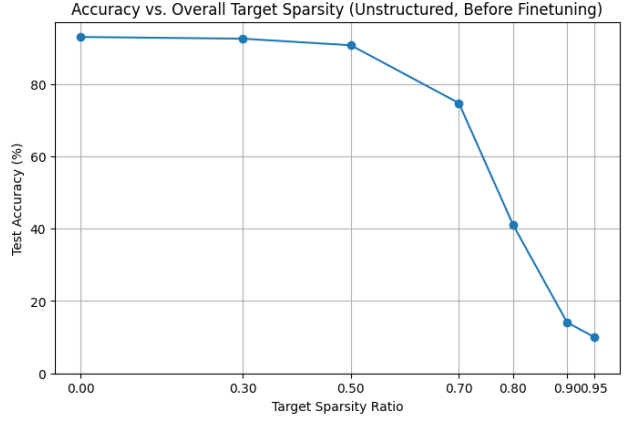


Figure 6. As sparsity levels increase in unstructured pruning, accuracy remains relatively stable up till 0.50 sparsity, after which it starts to drop more significantly. This is mitigated by fine-tuning the model post-pruning.

provides immediate size benefits, with INT4 achieving an 80% reduction. QAT size reductions are realized upon deployment serialization.

Latency revealed the most significant difference. QAT models were clear winners on GPU, with ‘QAT-INT8’ and ‘QAT-INT4’ achieving a **20.5% speedup**. Conversely, ‘PTQ-INT8’ and ‘PTQ-BF16’ were substantially slower than the baseline on GPU. ‘PTQ-INT4’ showed outstanding performance on CPU (0.232 ms), highlighting its suitability for CPU-bound inference.

Table 3. Comparison of PTQ and QAT models. Latency was measured on a CUDA device unless specified as CPU.

METHOD	MODEL	BITS	ACC (%)	LATENCY (MS)	SIZE (MB)
FP32 BASELINE		32	60.91	2.415	516.71
4*PTQ	PTQ-FP16	16	60.91	2.142	258.36
	PTQ-INT8	8	60.90	3.719	156.97
	PTQ-INT4	4	61.00	0.232	104.42
	PTQ-BF16	8	60.90	4.084	258.36
4*QAT	QAT-FP16	16	60.53	1.915	258.36
	QAT-BF16	16	61.38	3.795	258.36
	QAT-INT8	8	60.62	1.921	516.71
	QAT-INT4	4	60.24	1.925	516.71

3.2.2. SCALING LAW ANALYSIS

The relationship between model accuracy and bit-width precision for both PTQ and QAT is visualized in Figure 7. This analysis reveals how performance scales as the model is compressed into lower-precision formats.

Several key trends can be observed from the plot:

Exceptional Stability of PTQ: The accuracy of Post-

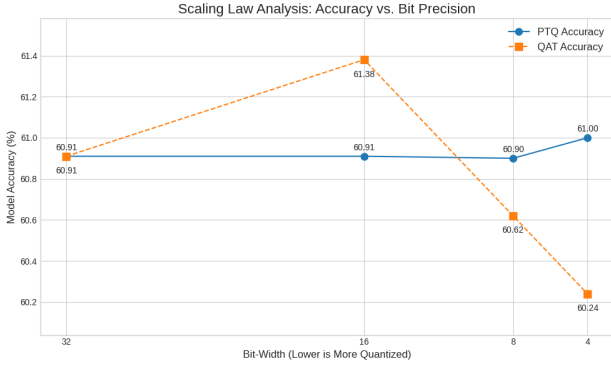


Figure 7. Scaling Law Analysis: Accuracy vs. Bit Precision for PTQ and QAT. The y-axis is zoomed to highlight minor differences in accuracy.

Training Quantization (blue line) is remarkably stable across all bit-widths. Accuracy remains almost perfectly flat, starting at 60.91% for the 32-bit baseline, holding at 60.91% for 16-bit, and dipping negligibly to 60.90% at 8-bit. Surprisingly, the 4-bit PTQ model achieves a slightly higher accuracy of 61.00%. This indicates that for the VGG-11 architecture, PTQ does not degrade accuracy and is a highly robust compression technique.

Variable Performance of QAT: In contrast, the accuracy of Quantization-Aware Training (orange dashed line) shows significant variation.

- At 16-bits, the QAT model’s accuracy peaks at **61.38%**, substantially outperforming both the baseline and the PTQ model. This suggests that the QAT process, specifically with the BFloat16 format used in our experiments, may have a regularizing effect that improves the model’s generalization.
- Below 16-bits, the QAT model’s accuracy steadily declines, dropping to 60.62% at 8-bits and further to 60.24% at 4-bits.

3.2.3. MIXED-PRECISION PERFORMANCE

The results for our three mixed-precision configurations are shown in Table 4. All methods successfully preserved the baseline accuracy. The ‘Simple’ heuristic provided the best balance, improving latency by 16.8% over the FP32 baseline. The ‘Adaptive’ method suffered from extremely high latency, likely due to framework overhead from breaking layer fusion opportunities.

3.2.4. THE EFFECT OF OUTLIERS

This task investigates how extreme values (outliers) in weight and activation distributions affect quantization qual-

Table 4. Performance of Mixed-Precision Configurations.

CONFIGURATION	BITS	ACC (%)	LATENCY (MS)
UNIFORM (PTQ-INT8)	8	60.90	3.719
SIMPLE (FIRST/LAST)	8/32	60.91	2.009
ADAPTIVE (VARIANCE)	8/32	60.91	7.826

ity. The performance of different outlier-handling strategies is summarized in Table 5, and the layer-wise quantization error for each strategy is visualized in Figure 8.

At 8-bit precision, the impact of outliers is minimal. Table 5 shows that ‘Uniform-8bit’ and ‘Log-8bit’ quantization achieve near-baseline accuracy. The ‘Mean Quantization Error’ for these methods is extremely low across all layers, as seen in Figure 8. Aggressive clipping (‘99.9%’) slightly harms accuracy, which corresponds to the introduction of a high ‘Maximum Quantization Error’ for the clipped values.

At 4-bit precision, the management of outliers becomes critical.

- **Uniform-4bit quantization** leads to a catastrophic **24% drop in accuracy**. The left plot in Figure 8 shows why: this method (purple bars) results in a very high ‘Mean Quantization Error’ across all layers. With only 16 quantization levels, the presence of outliers stretches the quantization range so widely that the representation for the bulk of the data is severely degraded.
- **Clipped-4bit-99.9% quantization**, in contrast, recovers nearly all of the performance, with only a 4.48% drop in accuracy. This success is directly explained by the error plots. While its ‘Maximum Quantization Error’ is high (right plot, brown bars)—a direct result of clipping the extreme values—its ‘Mean Quantization Error’ is significantly lower than that of the uniform method.

The analysis of the layer-wise error in Figure 8 provides a crucial insight: mean quantization error, not maximum error, is the better correlate of final model accuracy. The clipping strategy deliberately accepts a large maximum error on a tiny fraction of outlier data points in order to drastically reduce the mean error for the vast majority of values. This trade-off is the key to successful low-bit quantization.

Furthermore, the plots show that quantization error is not uniform across the network. Deeper layers, such as `features.11`, tend to exhibit higher mean errors in the 4-bit regime, suggesting they might be more sensitive to precision loss. This observation reinforces the motivation for mixed-precision approaches, where such sensitive layers could be assigned a higher bit-width.

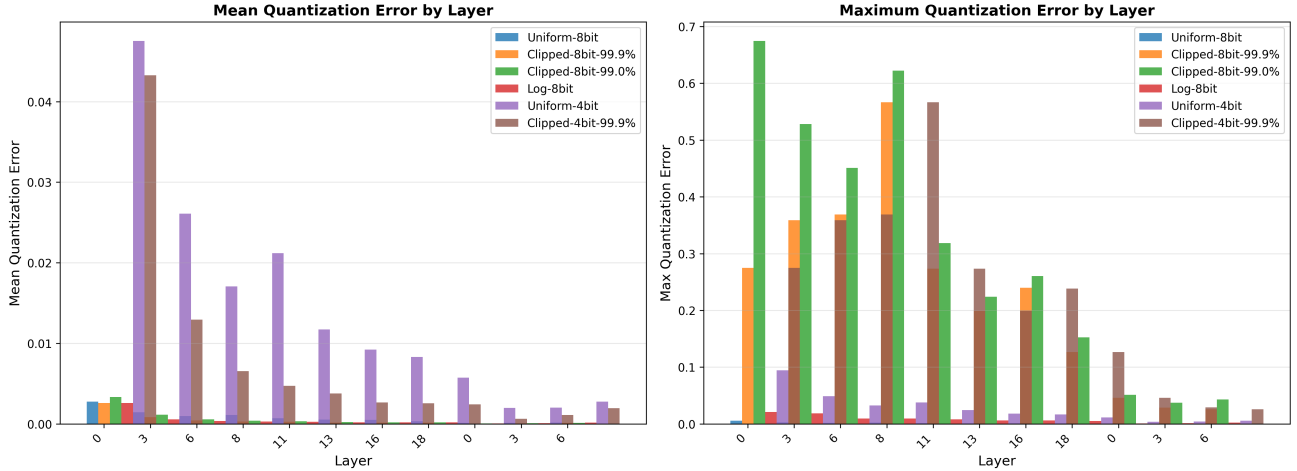


Figure 8. Mean and Maximum Quantization Error per Layer for Different Outlier-Handling Strategies. The left plot shows that 4-bit uniform quantization induces a high mean error, which is effectively reduced by clipping. The right plot shows that clipping strategies inherently have a high maximum error, as they sacrifice the fidelity of a few outlier values.

Table 5. Performance of outlier-aware quantization strategies.

STRATEGY	BITS	ACC (%)	Δ ACC	MEAN ERR	MAX ERR
BASELINE	32	60.91%	-	-	-
UNIFORM-8BIT	8	60.83%	-0.08%	0.00079	0.00159
CLIPPED-99.9%	8	59.46%	-1.45%	0.00047	0.22811
LOG-8BIT	8	60.78%	-0.13%	0.00045	0.00809
UNIFORM-4BIT	4	36.90%	-24.01%	0.01396	0.02710
CLIPPED-4BIT-99.9%	4	56.43%	-4.48%	0.00751	0.22807

3.3. Knowledge Distillation

We present the empirical results from our series of KD experiments. The VGG-16 teacher (T) achieved a baseline accuracy of 74.00% on the test set.

3.3.1. LOGIT MATCHING VARIANTS

The three output-based methods exhibited distinct performance characteristics, as shown in Table 6. Basic Logit Matching emerged as the strongest performer, with Decoupled KD trailing marginally. Label Smoothing, operating without teacher guidance, yielded the lowest accuracy, confirming its role as a regularizer rather than a distillation method.

METHOD	TEST ACCURACY (%)
BASIC LOGIT MATCHING (LM)	62.96
LABEL SMOOTHING (LS)	60.76
DECOUPLED KD (DKD)	62.52

Table 6. Test accuracy of VGG-11 student trained with different logit-based methods.

3.3.2. COMPARATIVE SOTA ANALYSIS

Table 7 reveals substantial variation across KD paradigms. Hint-based distillation demonstrated the most pronounced improvement, followed by Logit Matching. CRD’s marginal gain suggests limited efficacy in this experimental configuration.

METHOD	VAL. ACCURACY (%)	IMPROVEMENT
S_I	58.36	-
S_{LM}	61.22	+2.86%
S_{Hint}	63.20	+4.84%
S_{CRD}	58.74	+0.38%

Table 7. Validation accuracy of VGG-11 student models trained with different SoTA KD methods, compared to the independent student (S_I).

3.3.3. PROBABILITY DISTRIBUTION ALIGNMENT

Analysis of distribution alignment using KL divergence on standard softmax outputs ($T = 1$) revealed nuanced patterns across methods, as detailed in Table 8. Hint-based distillation achieved the lowest average divergence, followed by the independent student, CRD, and LM. Counterintuitively, Hints outperformed even the baseline, while LM exhibited the highest divergence despite strong classification performance. These large absolute values stem from computing divergence on sharp, untamped distributions rather than the soft targets used during training ($T = 4.0$). The high correlation between KL and JS metrics confirms measurement consistency, though the inverse relationship between divergence and accuracy for LM suggests distribution mimicry alone does not guarantee performance gains.

METHOD	KL DIV.	JS DIV.	CORR.
S_{LM}	11.62	2.01	0.82
S_{Hint}	8.23	1.81	0.79
S_{CRD}	10.40	1.96	0.77
S_I (BASELINE)	9.96	1.95	-

Table 8. Average KL and JS divergence between teacher and student probability distributions, with KL-JS correlation for each method. Lower divergence indicates closer alignment.

3.3.4. LOCALIZATION KNOWLEDGE TRANSFER

Figure 9 presents GradCAM visualizations across all models. The quantitative metrics in Table 9 reveal that S_{LM} and S_{Hint} achieve high structural similarity with the teacher’s attention patterns, with S_{Hint} exhibiting the tightest alignment. In contrast, S_{CRD} and S_I demonstrate substantially divergent localization behavior.

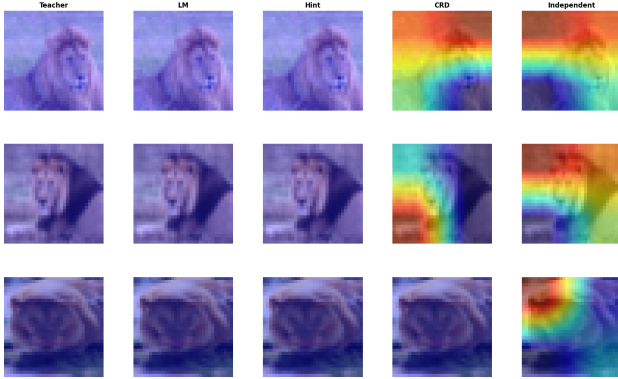


Figure 9. GradCAM visualizations comparing the localization patterns of the Teacher (T), distilled students (S_D), and the independent student (S_I). The S_{LM} and S_{Hint} models demonstrate a clear ability to mimic the teacher’s focus.

MODEL VS. TEACHER	SSIM ($\uparrow$)	MSE ($\downarrow$)
S_{LM}	0.8947	0.1053
S_{Hint}	0.9244	0.0756
S_{CRD}	0.7464	0.2536
S_I (BASELINE)	0.6386	0.3614

Table 9. Quantitative comparison of GradCAM heatmap similarity between the Teacher (T) and student models.

3.3.5. COLOR INVARIANCE DISTILLATION

Despite the teacher’s modest invariance gap, as detailed in Table 10, the distilled S_{CRD} student exhibited a substantial performance degradation on color-jittered data. This gap closely approximated that of the baseline and other KD methods, indicating failure to ambiently transfer invariance properties.

MODEL	STANDARD	JITTERED	GAP
T_{color} (TEACHER)	66.20%	63.70%	2.50%
S_{CRD} (FROM T_{color})	59.54%	50.04%	9.50%
S_I (BASELINE)	60.26%	50.38%	9.88%
S_{LM} (FROM T_{std})	62.96%	52.94%	10.02%
S_{Hint} (FROM T_{std})	64.02%	54.14%	9.88%

Table 10. Performance on standard vs. color-jittered validation sets to test for ambiently distilled invariance.

3.3.6. TEACHER MODEL CAPACITY

Table 11 presents results contradicting the hypothesis that larger teachers yield superior students. The VGG-19 teacher, despite comparable accuracy to VGG-16, produced an inferior student with a wider performance gap.

MODEL	VGG-16	VGG-19
TEACHER ACCURACY	74.00%	73.85%
STUDENT ACCURACY	62.96%	62.37%
PERF. GAP (T-S)	11.04%	11.48%

Table 11. Effect of teacher model capacity on student (VGG-11) performance using Logit Matching.

4. Discussion

4.1. Optimal Brain Surgeon: Pruning

Our results align with the fundamental assumption that neural networks are often over-parameterized (LeCun et al., 1989; Han et al., 2015), allowing for significant pruning without substantial loss in performance. The unstructured pruning approach effectively identifies and removes redundant weights, maintaining high accuracy even at elevated sparsity levels. This suggests that many weights in the network contribute minimally to the overall function, and their removal does not significantly impact the model’s predictive capabilities (i.e. the discriminative features are still being captured).

On the other hand, structured pruning demonstrates a more pronounced drop in model accuracy (partly because of a more aggressive reduction in model complexity), indicating that entire channels may contain important features that are critical for accurate predictions. However, the significant reduction in model size and inference latency achieved through structured pruning highlights its potential for deployment in resource-constrained environments.

Furthermore, the sensitivity analysis reveals that different layers exhibit varying degrees of tolerance to pruning. In our case, the latter layers were generally less sensitive to pruning, enabling higher sparsity levels in these layers without significant accuracy degradation. The reason earlier layers

may be more sensitive is that they may capture more fundamental features (like edges and textures) that are essential for the model’s overall performance.

The most significant insights were from our Grad-CAM visualizations and statistical comparisons. The visualizations showed that all models successfully identified the ”frog”, indicating that core feature extraction capabilities were preserved post-pruning. A more focused attention map in pruned models suggests that the network learned a more efficient representation of the input features. Moreover, the latencies observed in Table 2 were averaged over 10 inference runs. The VGG11-SBP model’s lower latency indicates that structured pruning creates a physically smaller, ”dense” model which is highly optimized to process inputs. The VGG11-UBP, despite having fewer parameters, still retains the original architecture’s complexity, leading to slightly higher latency.

4.2. Quantization

Our results highlight that the optimal quantization strategy is a nuanced choice dependent on the specific goals and constraints of deployment.

QAT for Latency vs. PTQ for Simplicity. There is a clear distinction between the methods. QAT is superior for minimizing latency on target hardware like GPUs, as it enables kernel fusion and other optimizations. The increased latency of PTQ-INT8 on GPU suggests that on-the-fly quantization overhead outweighed the benefits of integer arithmetic. PTQ, however, remains invaluable for its simplicity and effectiveness in reducing model size, especially for CPU deployment where its performance can be exceptional.

Hardware-Specific Performance. The stark contrast between GPU and CPU performance for low-bit PTQ is significant. This underscores that quantization performance is not universal but is tightly coupled to the target hardware architecture and its associated software stack.

The Critical Role of Outlier Mitigation. Our analysis confirms that as precision decreases, managing the quantization range becomes paramount. The success of ‘Clipped-4bit’ shows that sacrificing a tiny fraction of outlier values is a worthwhile trade-off to preserve the model’s integrity. This principle is crucial for deploying large models like Transformers, which are known to exhibit challenging outlier behavior.

Practical Implications of Mixed-Precision. While adaptive methods are theoretically appealing, the ‘Simple’ heuristic of keeping the first and last layers in high precision provided a better balance of accuracy and latency. This suggests that for mixed-precision to be effective for latency, the placement of high-precision layers must align with the compiler’s ability to perform optimizations like layer fusion.

4.3. Knowledge Distillation

The dominance of Hint-based distillation in both accuracy and localization transfer suggests that intermediate feature matching provides a more effective knowledge transfer mechanism than output-level mimicry alone. This aligns with the theoretical framework of FitNets (Romero et al., 2015), where forcing structural alignment in hidden representations enables the student to internalize the teacher’s hierarchical feature extraction process. The strong performance of basic Logit Matching, while less pronounced, confirms Hinton et al.’s (Hinton et al., 2015) original insight that soft targets encode rich inter-class relationships beyond hard labels.

The GradCAM analysis provides compelling evidence that KD transfers not merely predictive capacity but also the teacher’s attentional strategy. The high SSIM scores for S_{LM} and S_{Hint} indicate these methods successfully replicate spatial localization patterns, corroborating findings by Ojha et al. (Ojha et al., 2023) that visual grounding is a fundamental component of distilled knowledge. Conversely, CRD’s poor localization alignment, despite its contrastive objective, suggests that global representation matching may sacrifice spatial correspondence. This could explain CRD’s underwhelming accuracy gains in our experiments—the method may be optimizing for a form of similarity that is less relevant for discriminative vision tasks on small-scale datasets like CIFAR-100.

The color invariance experiment yielded a negative result, with S_{CRD} failing to inherit the teacher’s robustness to chromatic perturbations. This outcome challenges the hypothesis that invariances can be ambiently distilled through representation alignment alone. Tian et al. (Tian et al., 2022) proposed CRD for capturing instance-level relationships, but our results suggest this does not extend to transferring transformation invariances without explicit exposure during student training. This distinction is critical: while KD can transfer learned patterns within the data manifold, properties that depend on augmentation-induced regularization may require the student to experience those augmentations directly, or alternatively, necessitate invariance-aware distillation objectives.

The teacher capacity experiment provides empirical support for Cho and Hariharan’s (Cho & Hariharan, 2019) capacity gap hypothesis. The larger VGG-19 teacher, despite near-identical accuracy to VGG-16, produced a marginally worse student. This phenomenon can be attributed to a mismatch in representational complexity: when the teacher’s capacity vastly exceeds the student’s, the student may struggle to compress the teacher’s nuanced decision boundaries into its limited parameter space, leading to inefficient or incomplete knowledge transfer. This finding has practical implications—practitioners should consider the student’s

capacity when selecting a teacher, rather than defaulting to the largest available model.

4.4. Holistic Comparison

When comparing the three compression techniques, we reflect on the following:

- **Convenience and Practicality:** PTQ is the most convenient method. A "plug-and-play" post-processing step with no retraining required. Pruning, QAT and KD either require full training or fine-tuning, with pruning demanding a staged pipeline of sensitivity scans, channel ranking, and desired sparsity. Standard Logit Matching (KD) is relatively straightforward, often just involving an additional loss term.
- **Potential Drawbacks and Advantages:** KD was the only method capable of improving accuracy over a baseline. However, our VGG11-SBP displayed a significant latency reduction by physically reducing the model complexity. And, quantization techniques provided significant GPU speedups. However, these methods had their own share of problems. Low-bit quantization was severely vulnerable to outliers, and unstructured pruning failed to give a significant inference time reduction.
- **Effectiveness vs. Budget:** With a fixed, small architecture, our results indicate that KD may be the optimal choice as it enhances a model's performance without altering its size. Pruning and Quantization aim to preserve accuracy while shrinking the model; ideal for minimal inference or training time. However, on large scale models, pruning becomes cumbersome, and it may be more effective to quantize such models.

Ideally, we may want to combine these in a manner that brings out the best of each technique. KD and pruning can be used in tandem to generate a smaller, efficient model (pruned) that has learned features from a complex teacher model. In some cases (large-language models), it may not be ideal to prune due to the complexity of the model layers and channels, hence, we can opt to quantize such distilled models.

5. Conclusion

This paper provided a comparative study of pruning, quantization, and KD as primary methods for model compression.

Through controlled experiments on the VGG11 architecture using the CIFAR-10 dataset, we observed that both, unstructured and structured pruning techniques are viable methods for model compression. Unstructured pruning effectively reduces the number of parameters while maintaining high

accuracy, whereas structured pruning offers significant reductions in model size and inference latency, albeit with a more pronounced accuracy drop that can be mitigated through fine-tuning.

We found that Quantization-Aware Training is the most effective method for reducing GPU inference latency, achieving a 20.5% speedup while maintaining accuracy. PTQ offers a simpler path to significant model size reduction, with 4-bit PTQ being particularly potent for CPU deployments. Furthermore, our analysis demonstrates that outlier clipping is a critical technique for preserving accuracy in low-bit (4-bit) quantization regimes. These findings underscore that the optimal quantization strategy depends on a careful consideration of the trade-offs between accuracy, model size, latency, implementation complexity, and the specific characteristics of the deployment hardware.

The experimental setup for KD revealed that the choice of distillation technique is highly consequential. FitNet demonstrated superior performance in terms of raw accuracy and localization transfer, while CRD and LM had varied success. We also empirically confirmed that a larger teacher does not guarantee a better student, highlighting the importance of managing the "capacity gap" between models.

Ultimately, the optimal compression strategy depends on the specific constraints of the application, and a synergistic combination of these techniques often provides the best path to efficient, high-performance deep learning.

References

- Cho, J. H. and Hariharan, B. On the efficacy of knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4794–4802, 2019.
- Han, S., Pool, J., Tran, J., and Dally, W. J. Learning both weights and connections for efficient neural networks, 2015. URL <https://arxiv.org/abs/1506.02626>.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- LeCun, Y., Denker, J., and Solla, S. Optimal brain damage. In Touretzky, D. (ed.), *Advances in Neural Information Processing Systems*, volume 2. Morgan-Kaufmann, 1989. URL https://proceedings.neurips.cc/paper_files/paper/1989/file/6c9882bbac1c7093bd25041881277658-Paper.pdf.
- Ojha, U., Li, Y., Rajan, A. S., Liang, Y. J., and Lee, Y. J.

What knowledge gets distilled in knowledge distillation?
arXiv preprint arXiv:2306.05602, 2023.

Romero, A., Ballas, N., Kahou, S. E., Chassang, A., Gatta, C., and Bengio, Y. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2015.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128(2):336–359, October 2019. ISSN 1573-1405. doi: 10.1007/s11263-019-01228-7. URL <http://dx.doi.org/10.1007/s11263-019-01228-7>.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.

Tian, Y., Krishnan, D., and Isola, P. Contrastive representation distillation. In *International Conference on Learning Representations (ICLR)*, 2022.

Zhao, B., Cui, Q., Song, R., Qiu, Y., and Liang, J. Decoupled knowledge distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11953–11962, 2022.

Appendix

A. Additional Experimental Results

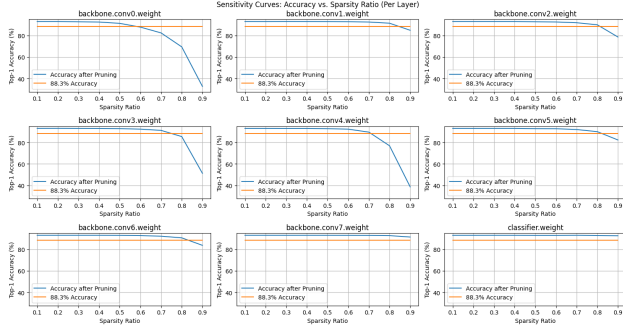


Figure 10. Sparsity scans of all layers before final pruning is performed.

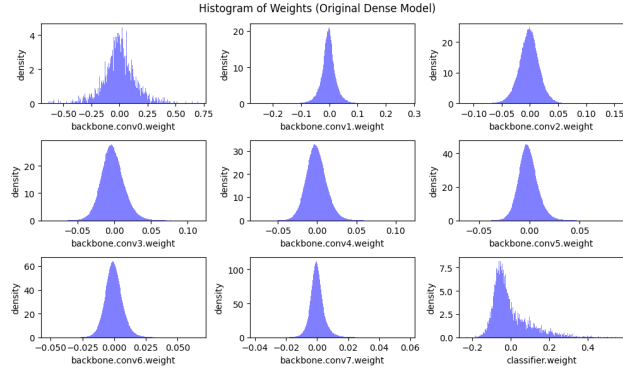


Figure 11. Weight Distributions of all layers before any pruning is performed.

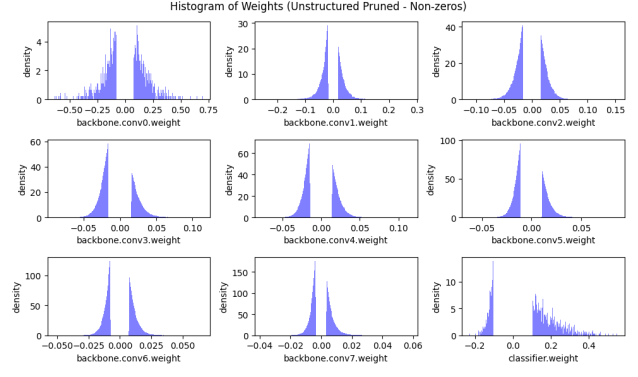


Figure 12. Weight Distributions of all layers after Unstructured Pruning.

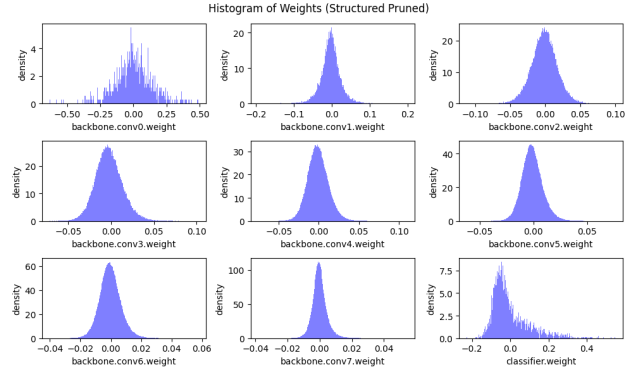


Figure 13. Weight Distributions of all layers after Structured Pruning.

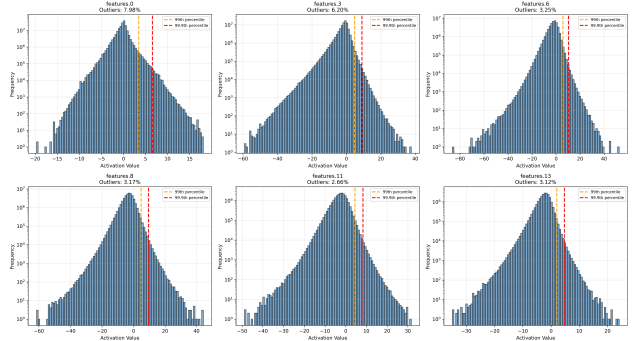


Figure 14. Activation Distributions for Selected Layers. The presence of outliers is evident, particularly in deeper layers, which justifies the need for outlier-aware quantization strategies.

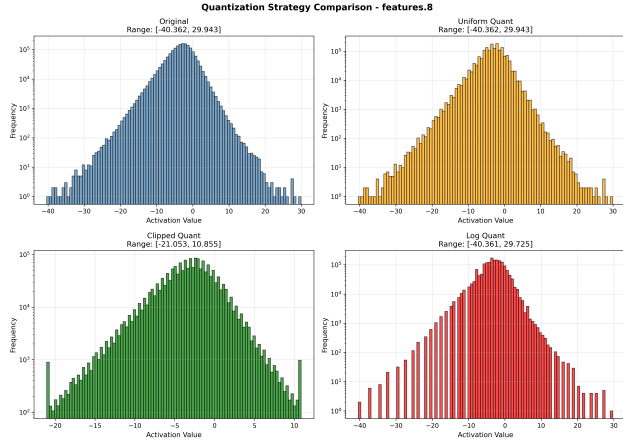


Figure 15. Comparison of Different Quantization Strategies on Weight Distributions. The uniform quantization shows significant distortion in the presence of outliers, while clipped and logarithmic methods better preserve the core distribution.

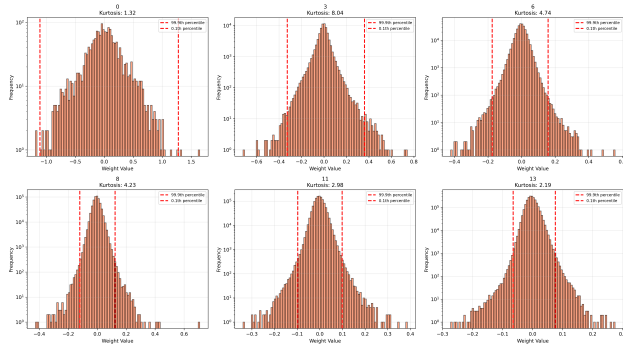


Figure 16. Weight Distributions for Selected Layers. The distributions exhibit long tails, indicating the presence of outliers that can adversely affect quantization quality.